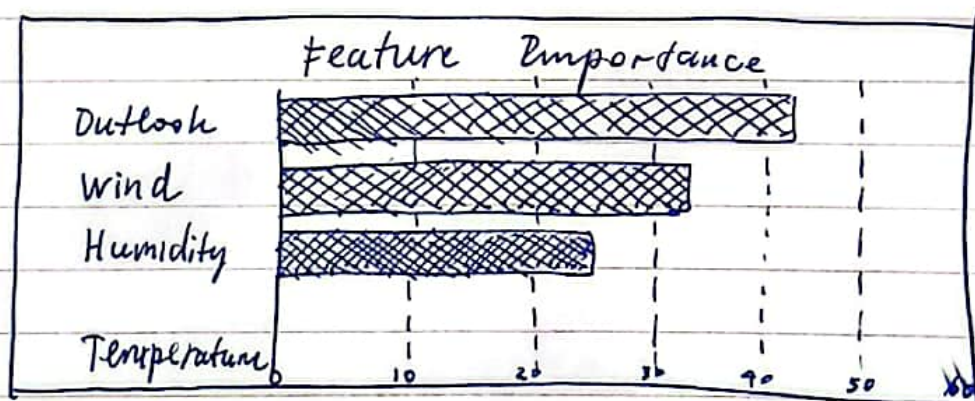


8 - dars : Feature Importance

FUTURE IMPORTANCE

- ★ FI xar bir feature umumiy model "predictionga" xisla qo'shganligini o'lchaydi.
- ★ model.feature-importances_ --- tree based
- ★ Coefficients --- linear models



LIMITATIONS

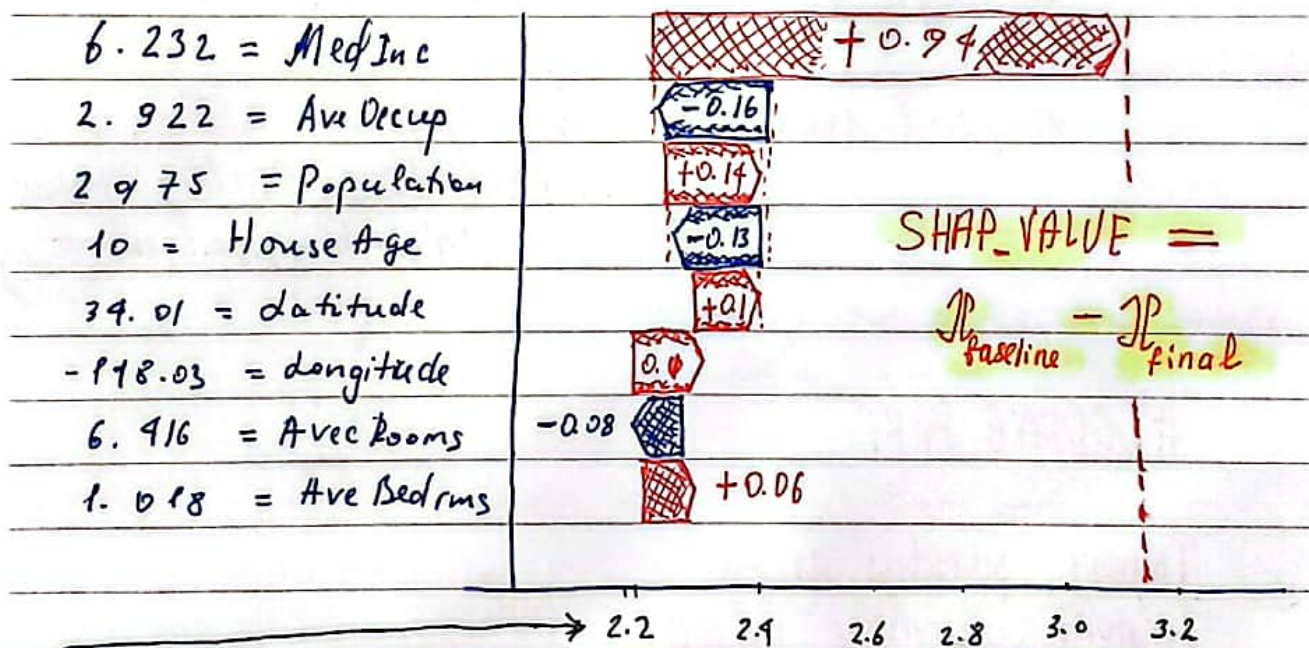
- ★ Feature qanday ta'bir qildi (positive yoki negative)
- ★ Qaysi qiymat yoki qatorda (sample) "outputga" ta'bir qilganini ko'rsatmaydi.
- ★ Highly correlated bo'lgan payfa FI yaxshi ishlamay qoladi.

SHAP VALUES

★ SHAP (SHapley Additive exPlanations)

★ SHAP VALUE --- Global feature importance

★ SHAP VALUE --- input qiymat/ni qanchalik model predictionni baseline qiymatdan farqi tomonga sura olishga qo'shgan hissalarni o'lchaydi.



shap valueuning asl maqsadi $\Rightarrow$ Input qiymat/ "Baseline" qiymatni, qancha va farqi tomonga suradi.

$\rightarrow 2.2 + 2.4 + 2.6 + 2.8 + 3.0 + 3.2 = 16.2 \Rightarrow$ danashu final "prediction"

* SHAP (SHAPLEY ADDITIVE EXPLANATIONS)

* explanatory, not evaluate

* Example (player rating)

* +3 - - - yawschi shot

* -2 - - - yomon pass

* 0 - - - assists

* Positive SHAP $\rightarrow$ feature pushes rating up

* Negative SHAP $\rightarrow$ feature pushes rating down

* Yawschi yoki yomon feature maqsadga ko'ra aniqlanadi:

* high rating $\rightarrow$ positive influence is helpful

* find weaknesses $\rightarrow$ negative influence is informative. (salbiy $\Rightarrow$ teskari ta'sir ham foydali ma'lumat beradi)

CODING PART

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import r2_score, mean_squared_error

import shap
import matplotlib.pyplot as plt
```

- Copy of my df - -

df-copy = df.copy()

- - - Feature and target - -

X = df-copy.drop(columns=["X6"], errors="ignore")

y = df-copy["X6"]

- - - Identify numeric and categorical features - -

numeric-features = X.select_dtypes(include=["int64",
"float64"]).columns

categorical-features = X.select_dtypes(include=["object"]
.columns.

- - - Simple label encoding for categorical inside
a function - - -

def label_encode_df(df, cat_cols):

df-encoded = df.copy()

encoders = {}

for col in cat_cols:

le = LabelEncoder()

df-encoded[col] = le.fit_transform(df-encoded
[col].astype(str))

encoders[col] = le

return df-encoded, encoders

X-encoded, encoders = label_encode_df(X, categorical-features)

- - - Train / Test Split - - -

X-train, X-test, y-train, y-test = train-test-split(X-encoded, y,
random_state=0)

--- Pipeline with scaling numeric features only ---

```
preprocessor = ColumnTransformer ([
    ('num', StandardScaler(), numeric_features)
], remainder = "passthrough") # leave label-encoded
categorical as-is
```

```
overfit_model = Pipeline ([
    ("preprocess", preprocessor),
    ("regressor", DecisionTreeRegressor (max_depth =
    None, random_state = 42)) ])
```

--- Train model ---

```
overfit_model.fit (X_train, y_train)
```

--- Evaluate ---

```
y_train_pred = overfit_model.predict (X_train)
```

```
y_test_pred = overfit_model.predict (X_test)
```

```
print ("Train  $R^2$  :", r2_score (y_train, y_train_pred))
```

```
print ("Test  $R^2$  :", r2_score (
```

- SHAP explanation - - -

Preprocess test set

```
X_test_preprocessed = overfit_model.named_steps
["preprocess"].transform (X_test)
```

```
explainer = shap.TreeExplainer (overfit_model.
named_steps ["regressor"])
```

```
shap-values = explainer.shap-values(X-test-preprocessed)
```

```
# --- Waterfall plot for first sample ---
```

```
shap.waterfall-plot (shap.Explanation (
    values = shap-values [0],
    base-values = explainer.expected-value,
    data = X-test-preprocessed [0],
    feature-names = list(numeric-features) +
        list(categorical-features)
))
```

```
# --- Summary plot ---
```

```
shap.summary-plot (shap-values, X-test-preprocessed,
    feature-names = list(numeric-features) + list(
        categorical-features))
```

LINEAR REGRESSION UCHUN

```
import sklearn
import shap
```

```
# a classical housing price dataset
```

```
X, y = shap.datasets.california (n-points=1000)
X100 = shap.utils.sample (X, 100) # 100 instances
for use as the background distribution
```

a simple linear model

```
model = sklearn.linear.linear_regression()
```

```
model.fit(X, y)
```

```
print("Model coefficients:\n")
```

```
for i in range(X.shape[1]):
```

```
    print(X.columns[i], "=", model.coef_[i]  
          .round(5))
```

```
shap.partial_dependence_plot(
```

```
    "MedInC",
```

```
    model.predict,
```

```
    X[100],
```

```
    ice = False,
```

```
    model_expected_value = True,
```

```
    feature_expected_value = True,
```

```
)
```

Compute the SHAP values for the linear model

```
explainer = shap.Explainer(model.predict, X[100])
```

```
shap_values = explainer(X)
```

make a standard partial dependence plot

```
sample_id = 20
```

shap. partial-dependence-plot()

"Med Inc",

model.predict,

X100,

Bu yer fagat 100 ta gatorini olgan

model.expected-value = True,

feature.expected-value = True,

ice = ~~True~~ False,

shap-values = shap-values[sample-ind :
sample-ind + 1, :],

)

shap.plots.waterfall(shap-values[sample-ind],
max-display = 14)

Nima uchun foydali:

* Feature engineeringdagi yaratilgan feature-larni qanchalik foydali / foydasiz ekanligini ko'ra olamiz.

* Local explanations nima uchun model bunday prediction qildi

* Barcha modellar buni ishlay oladi.

Feature Importance vs SHAP values

Feature Importance = basic global interpretability
SHAP = deeper global + local interpretability

1. Feature Importance - "Qaysi feature muhim?"
↳ Model umumiy prediction/ola qaysi featurelardan ko'proq foydalanadi?

2. SHAP - "Qaysi feature muhim VA prediction qaysi tomonga suradi?"

↳ Model = boshlang'ich predictionni bitta bemor uchun:

"baseline probability $\approx 8\%$ "

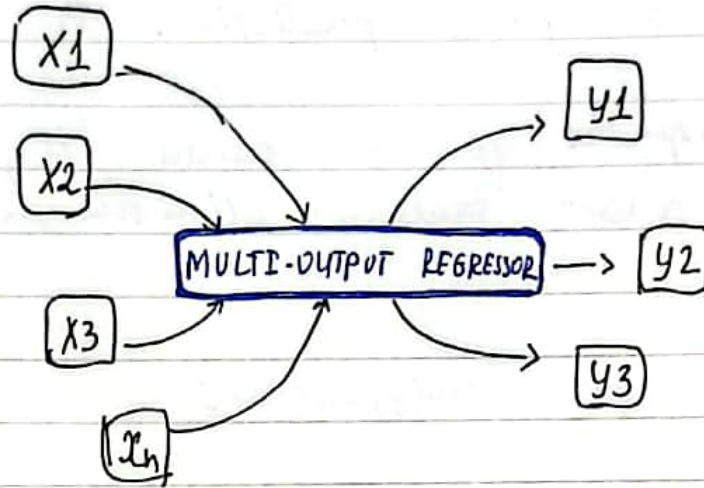
keyin:

"Age" $\rightarrow +6\%$

"Poor General Health" $\rightarrow +9\%$

~~Thesisga~~ Thesisga kiritib ketishim k-h.

MULTI-OUTPUT MODELS



MULTI-CLASS

Only one output class at a time } Class / multiple birlishi m-n ammo output 1ta chigadi.

MULTI-LABEL $\Leftrightarrow$ MULTI-OUTPUT

Can have multiple output classes at a time } Bunda jarabi bir nechta chigishi m-n

★ Multi-Output Regressor

★ Targets: Continuous (numeric) values

★ Example: Predict a soccer player's:

★ Goals

★ Assists

★ Scikit-learn: Use Multi Output Regressor

Notes :

- * Each target is numeric.
- * The model can capture correlations between targets to improve predictions.

* Multiple - outputni qo'llayot gauda bir-biriga yozgin "feature lar" ta'labini o'qini shi h-h.

Multi - Output Classification :

* Targets : Categorical values (binary or multi-class)

* Example : Predict a player's :

* Injury risk (Yes / No → binary)

* Player role (Forward / Midfielder / Defender → multi-class)

* Scikit - learn : Use **Multi Output Classifier**

* Notes :

* Each target can have different numbers of classes.

* If all outputs are **binary**, it is also called **multi-label classification** (special case)

CODING

```

import pandas as pd
import
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.multioutput import MultiOutputRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error,
r2_score

```

```

target_cols = ["goals", "xB"]

```

```

X = df.drop(columns = target_cols + ["id", "
player_name"])

```

```

y = df[target_cols]

```

```

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size = 0.2, random_state = 42)

```

```

categorical_features = ["position", "team_title"]

```

```

numerical_features = [col for col in X.columns
if col not in categorical_features]

```

```

# list comprehension (categorical_features & numerical)

```

```

numerical_transformer = Pipeline(steps = [
("scaler", MinMaxScaler())

```

```

])

```

categorical_transformer = Pipeline (steps = [
 ("encoder", OneHotEncoder(handle_unknown = "ignore"))
])

```
preprocessor = ColumnTransformer (
    transformers = [
        ("num", NumericTransformer, numeric-
          features),
        ("cat", categorical_transformer, categorical-
          features)
    ]
)
```

Multi output ni ishlatish uchun "baseline" algoritim tanlab olamiz
 base_model = Random Forest Regressor (random_state = 42)
 multi_output_model = MultiOutputRegressor (base_model)

base_model

```
pipeline = Pipeline (steps = [
    ("preprocessor", preprocessor),
    ("regressor", multi_output_model)
])
```

pipeline

```
pipeline.fit(X_train, y_train)
```

```
y_pred = pipeline.predict(X_test)
```

```
for i, target in enumerate(target_cols):  
    mse = mean_squared_error(y_test[target],  
                             y_pred[:, i])
```

```
    r2 = r2_score(y_test[target], y_pred[:, i])
```

```
    print(f"{target} → MSE: {mse:.4f}, R2: {r2:.4f}")
```

```
import joblib
```

```
model_filename = "multiooutput_goals_xG_model.joblib"
```

```
joblib.dump(pipeline, model_filename)
```

```
print(f"\n Model saved successfully as '{model_filename}'")
```

```
user_input = [
```

```
{
```

```
    "id": 1,
```

```
    "player_name": "Erling Haaland",
```

```
    "games": 30,
```

```
    :
```

```
},
```

```
{
```

```
    "id": 2,
```

```
    :
```

```
}]
```



```
df_results = df[["id", "player-name"]].copy()
df_results["Predicted Goals"] = predictions[:, 0]
df_results["Predicted xG"] = predictions[:, 1]
```

```
print("\n Predictions for 3 players:")
print(df_results)
```

```
rmse_goals = np.sqrt(mean_squared_error(
    df_input['actual-goals'], df_results['Predicted-Goals']))
```

```
rmse_xG = np.sqrt(mean_squared_error(df_input[
    "actual-xG"], df_results["Predicted xG"]))
```

```
print(f"RMSE for Goals: {rmse_goals:.2f}")
print(f"RMSE for xG: {rmse_xG:.2f}")
```